

Product Requirement Document (PRD)

Project Build 5 (P005) – Case Study 2: Intelligent Travel Booking Assistant

System Identifier: `travel-booking-agent`

Total Evaluation Weight: 50 Marks

1. Business Objective & Background

The modern travel industry demands personalized, real-time, and highly secure customer service. The goal of this case study is to build an intelligent **Travel Booking Assistant Agent** using a tool-calling architecture.

Unlike a standard document-retrieval system, this agent must dynamically interact with external systems to serve the user. It will access a provided structured database to retrieve existing customer itineraries and seamlessly cross-reference that data with a live weather API to provide proactive, context-aware travel advice (e.g., *"I see you are flying to London next Tuesday; you should pack an umbrella as rain is forecasted."*).

2. Core Tech Stack Constraints

Participants must strictly adhere to the following infrastructure choices: * **Orchestration Framework:** `Pydantic AI` (Leveraging typed agent states, structured responses, and strict dependency injection). * **Database / SQL Layer:** `SQLite` (Using a standard DB driver/ORM to query the provided itinerary data). * **Weather API:** Any open REST API (e.g., *Open-Meteo*, which requires no API key, or *OpenWeatherMap*). * **Security & Safety Layer:** `Guardrails AI` (Implementing execution-stage guardrails using Profanity and Toxicity validators). * **Evaluation Framework:** `pytest` combined with `DeepEval` (for programmatic unit testing of agentic capabilities and LLM metrics).

3. Functional Requirements

3.1 Pydantic AI Agent & Tool Configuration

The core agent must be equipped with exactly **two tools** built using Pydantic AI's `@agent.tool` decorator paradigm: 1. **Database Connection Tool:** * Accepts user identifiers (e.g., Email or Booking ID) and retrieves upcoming travel plans, destination cities, travel dates, and hotel details from the provided `travel_data.db` file. 2. **Weather Information Tool:** * Accepts a location (city/coordinates) and dates, reaching out to an external Weather API to fetch current or forecasted weather conditions.

3.2 Dynamic Execution Logic

- The agent must interpret natural language prompts (e.g., *"What is the weather going to be like for my upcoming trip?"*) and deduce that it must *first* call the Database Tool to find out where the user is going, and *second* call the Weather Tool for that specific destination.
- The agent must synthesize the outputs of both tools into a natural, helpful, and concise conversational response.

4. Non-Functional Requirements

4.1 Security, Content Moderation & Input/Output Guardrails

Travel agents handle stressed customers; the application must be fortified against abuse using **Guardrails AI**. * **Input Guardrails:** Incoming user prompts must be scanned using **Profanity** and **Toxicity** validators. Angry or abusive prompts must be intercepted before triggering expensive LLM or API calls, returning a standardized polite de-escalation message. * **Output Guardrails:** The agent's final synthesized response must be passed through the same validators to ensure brand-safe, professional output under all circumstances.

4.2 Error Handling & Resilience

- The agent must gracefully handle scenarios where the booking ID does not exist in the database or the Weather API rate-limits/times out, informing the user without exposing raw system stack traces.

5. Automated Evaluation & Testing Suites

Participants must implement a robust test-driven development environment using **pytest** paired with **DeepEval**. A minimum of 10 distinct pytest functions must be written.

The tests must evaluate the agent's tool-augmented responses against the following metrics:

Metric	Evaluation Intent	Target Threshold
Answer Relevancy	Ensures the agent doesn't provide generic travel advice, but specifically answers the prompt based on the retrieved itinerary and weather data.	≥ 0.85
Contextual Precision	In a tool-use paradigm, the "context" is the JSON/data returned by the tools. Evaluates whether the LLM correctly isolated the right facts from the DB/API payloads.	≥ 0.80

6. Project Submission & Directory Standards

All development must take place on the participant's designated Git branch. The repository must reflect the exact structure below.

```
submissions/
└─ project-build/
    └─ pydantic-ai-application/
        └─ firstname-lastname/
            └─ travel-booking-agent/
                ├── db/                                # Must contain the provided database
                │   └─ travel_data.db
                ├── src/                                # Core application source code
                │   ├── agent.py                        # Pydantic AI agent definition
                │   ├── tools/                          # Tool implementations
                │   │   ├── weather.py
                │   │   └─ database.py
                │   └─ guardrails_config.py
                ├── tests/                              # pytest & DeepEval suites
                │   ├── conftest.py
                │   └─ test_agent_metrics.py
                ├── README.md                          # Setup, architecture, and run instructions
                └─ requirements.txt                    # Hardened dependency definitions
```

7. Detailed Evaluation & Marking Rubric

The maximum score achievable for this case study is **50 Marks**. Grading is divided into five distinct components:

Criteria	Marks	Description
Solution & Impact	10	Agent successfully chains DB retrieval and Weather API fetching to resolve user queries accurately.
Architecture & Eng.	10	Proper implementation of Pydantic AI tools, typed parameters, and clean API integration.
Documentation	10	Comprehensive <code>README.md</code> , strict typing (MyPy compliant), and clear inline comments.
Testing & Reliability	10	Clean integration of Guardrails AI. Successful <code>pytest</code> execution passing DeepEval thresholds.
Professionalism	10	Strict adherence to directory layouts, standard git history, and removal of API keys/secrets from code.

8. Development Resources & Setup Instructions

A. Database Setup (Provided): You have been provided with a mock SQLite database file named `travel_data.db`. You must place this file inside your `db/` directory. Your `database.py` tool must connect to this file and query it to retrieve user information.

Database Schema: The database contains a single table named `bookings` with the following columns: * `id` (INTEGER): Primary Key * `booking_id` (TEXT): The unique identifier for the trip (e.g., "TRV-101") * `user_name` (TEXT): The customer's full name * `user_email` (TEXT): The customer's email address * `destination` (TEXT): The city and country of the trip * `travel_dates` (TEXT): The duration of the trip * `hotel_details` (TEXT): The name of the booked accommodation

B. Free Weather API: To avoid managing API keys, participants are encouraged to use the **Open-Meteo API** (<https://open-meteo.com/>), which is entirely free for non-commercial use and does not require an authentication token. Participants will need to write a helper function to convert city names to latitude/longitude coordinates before querying the weather endpoint.